R.V.S SCHOOL OF ENGINEERING AND TECHNOLOGY

DINDIGUL

PROJECT TITLE: Multi-Line Insurance Policy and Claims Management System

TEAM LEADER : B. ISWARYA-
iswaryakodai@gmail.com

TEAM MEMBERS:

SUBITHIRA K- subisubithira91@gmail.com

MARIYAMMAL M -
mmariyammalmariyammal35@gmail.com

POORNIMA A -
poornipoornima544@gmail.com

SARAYA S-
saranyavengatesh846@gmail.com

Multi-Line Insurance Policy and Claims Management System – Salesforce Project

An insurancecarrier that manages vehicle, property and lifeinsurance currently relieson alegacy system, whichmakes policy quoting, issuance and claims handling manual, slow and inconsistent. The lack of a unifiedplatform makes cross-selling difficult, and manual claims review leads to long resolution times, highoperational cost and a poor customer experience.

This Salesforceproject provides a single platform that standardizes and accelerates quoting and policy issuance forallinsurance lines, automates claim initiation and routing based on policy type and claim severity, andgives a 360-degree view of the Customer, Policy and Claim data in one place.

Objectives

- Designaflexible data model that stores diverse product data (VIN for Vehicle, square footage for Property).
- ImplementScreen Flows that guide agents through multi-step quoting for different policy types.
- Developadynamic Apex/LWC Claims Dashboard so adjusters can manage their workload efficiently.
- Automateclaims approval/escalation by claim amount and policy history using Record-Triggered Flows.
- Enforce datavalidation across all insurance products for compliance and accuracy.

Challenges

- Variabledata management using Record Types and Field Sets.
- Approvalcomplexity: routing by policy type, claim amount and adjuster level.
- API integration simulation (Apex callout with mock test classes).
- User adoption through intuitive Screen Flows that minimise clicks.

Expected Benefits

Faster quote-to-policyturnaround, automatic claim routing to the right queue, controlled approval of high-value claims, state-based record visibility, and less administrative work for staff.

Creating Developer Account & Account Activation

Creating a Developer Org in Salesforce

- Go to https://developer.salesforce.com/signup
- On the sign-up form enter First name & Lastname, Email, Role: Developer, Company: College Name, Country: India, Postal Code, and a Username (in the format username@organization.com).
- Click Sign Me Up.

Account Activation

- Open the inbox of the email used to sign up and click Verify Account (the email may take 5–10 minutes).
- Set a password, answer the security question and click Change Password.

Data Management – Objects

Objects & Relationships

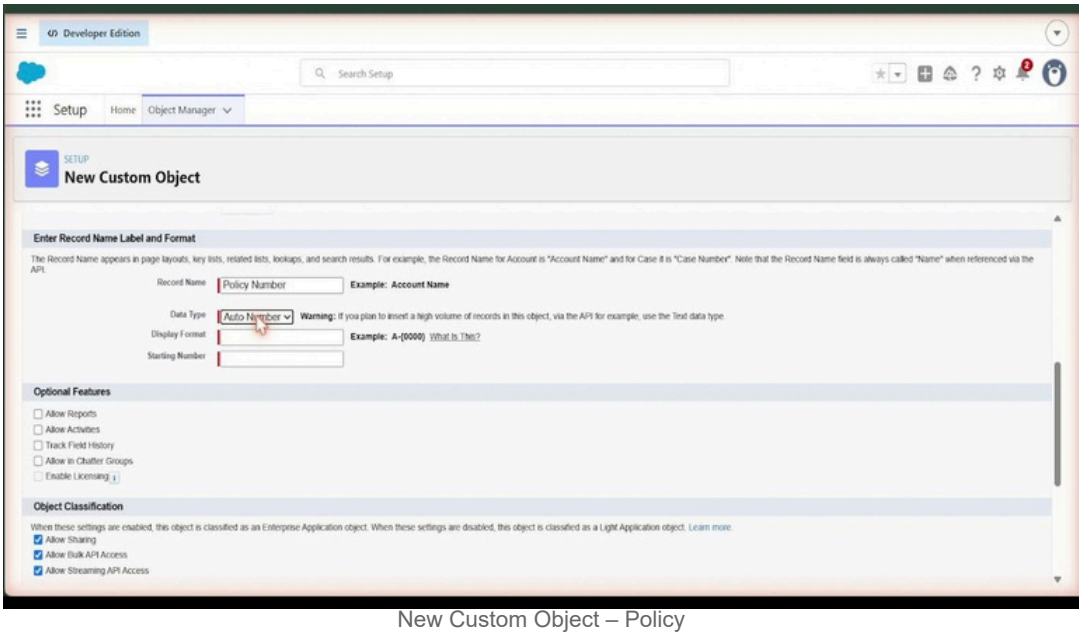
Object Name	Purpose	Relationships
Policy__c	Stores policy details for Auto, Property and Life	Lookup to Contact (Customer); parent of Claim
Claim__c	Tracks claims raised against a policy	Lookup to Policy, Lookup to User (Adjuster)

Milestone 1 – Create Policy Object

The purpose of the Policycustomobjectistoholdallinsurancepolicydatainone unified structure.

To create an object

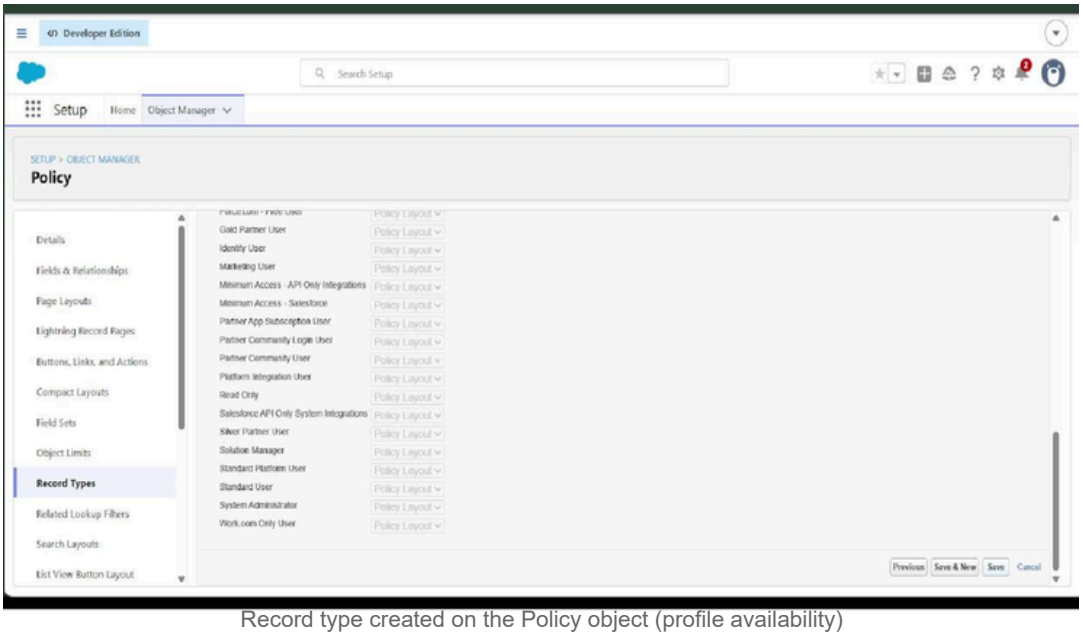
- Goto Setup → ObjectManager → Create → Custom Object.
- Label: Policy; PluralLabel: Policies; Object Name: auto-populated.
- Record Name: PolicyNumber; Data Type: Auto Number; Format: P-{0000}; Starting Number: 1.
- Optional features: Allow Reports, Allow Activities, Track Field History.
- Deployment Status:Deployed; Search Status: Allow Search → Save.



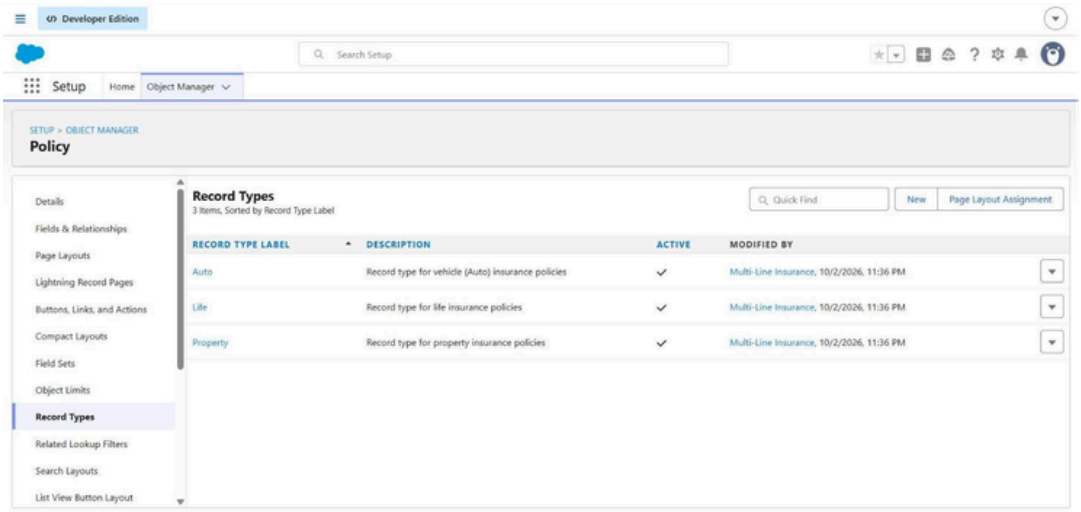
New Custom Object – Policy

Policy Record Types

- Object Manager → Policy__c → Record Types → New.
- Existing record type: Master; Record Type Label: Auto; add a description; check Active.
- Make available for all profiles → Next → leave page layout as is → Save.
- Repeat for Property and Life record types.



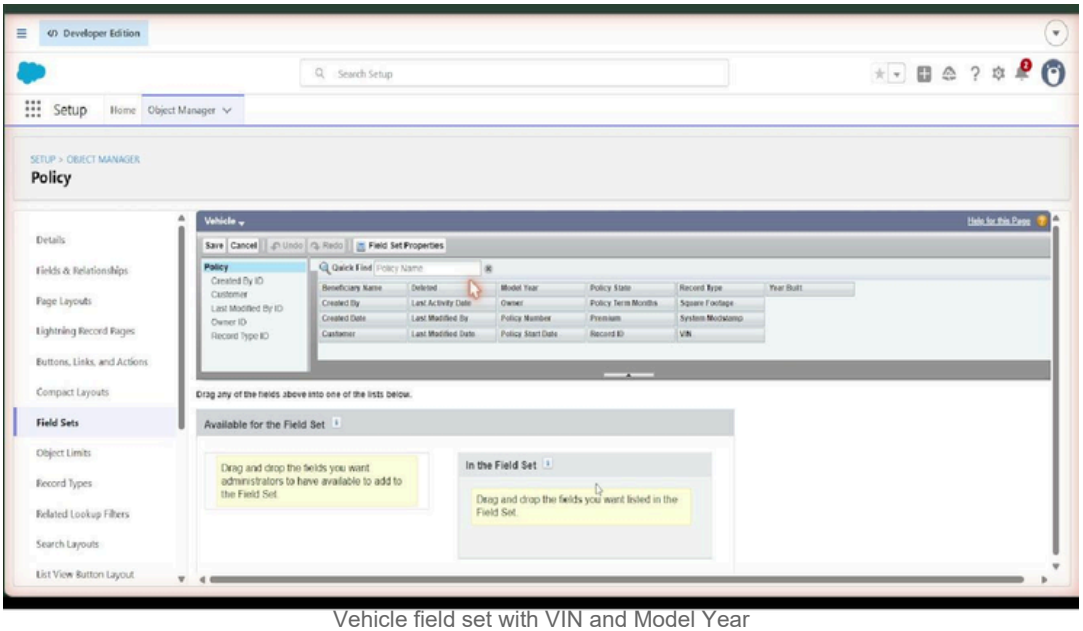
Record type created on the Policy object (profile availability)



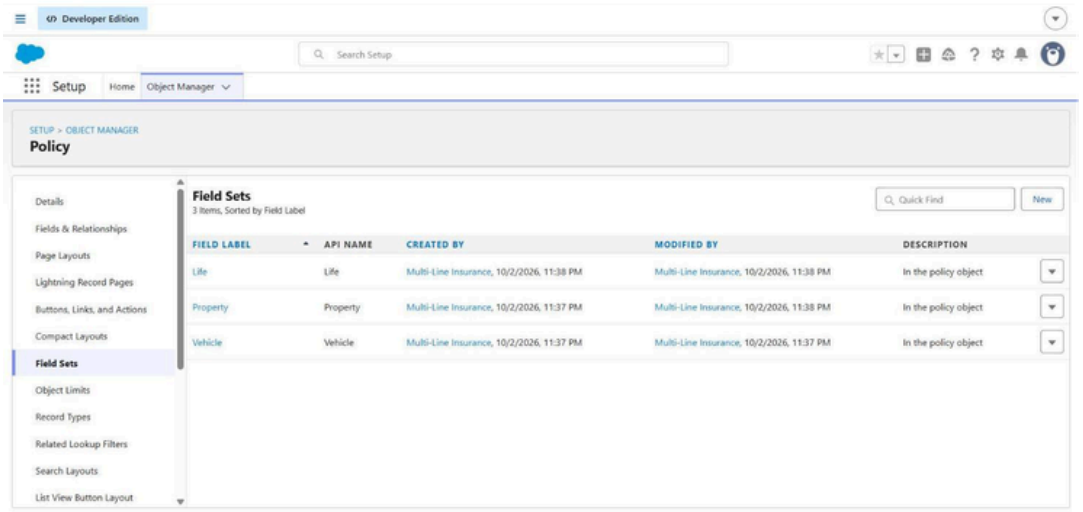
Policy record types: Auto, Life, Property

Policy Field Sets

- Object Manager → Policy__c → FieldSets→ New.
- Field Set Label: Vehicle (name auto-populated) → Save; drag VIN and Model Year into the field set → Save.
- Property field set: Square Footage, Year Built.
- Life field set: Beneficiary Name, Policy Term Months.



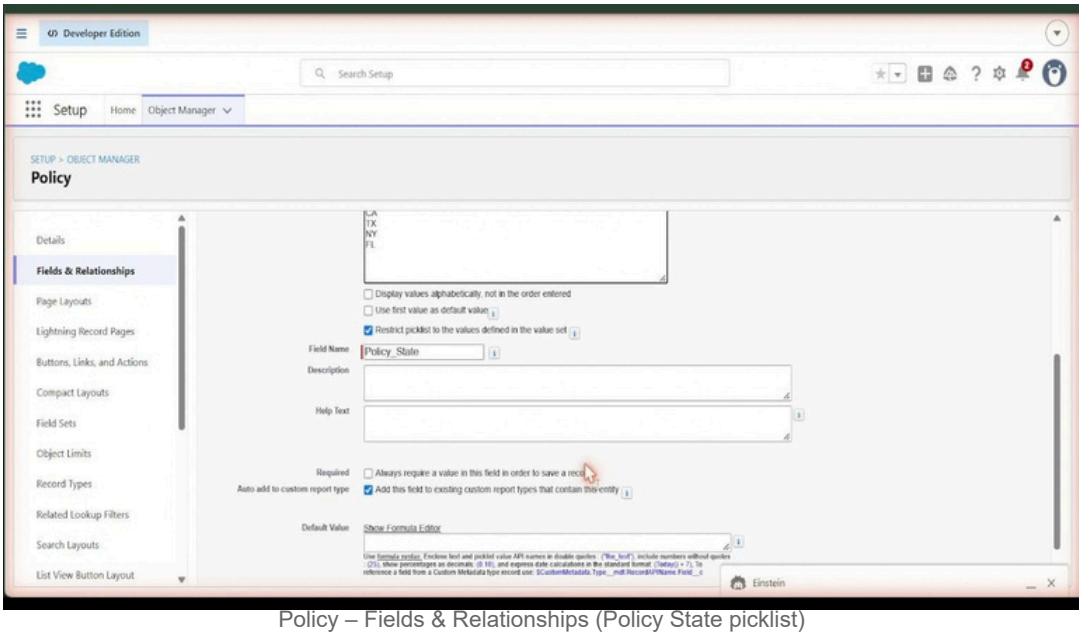
Vehicle field set with VIN and Model Year



Policy field sets: Life, Property, Vehicle

Fields in the Policy Object

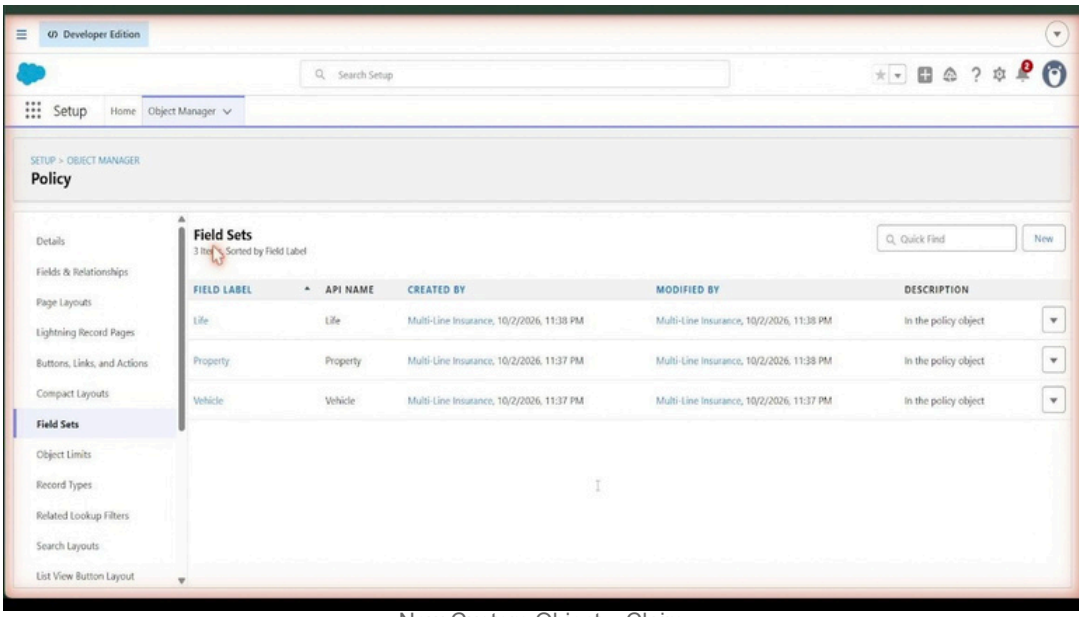
S.No	Field Name	DataType
1 2	BeneficiaryName	Text(50)
3 4	Customer	Lookup(Contact)
5 6	Model Year	Text(5)
7 8	Policy Start Date	Date
9	Policy Term Months	Number(18,0)
10	Premium	Currency(16,2)
	Square Footage	Number(18,0)
	VIN	Text(20)
	Year Built	Text(5)
	Policy State (used by the quoting flow and sharing)	Picklist(CA, TX, NY, FL)



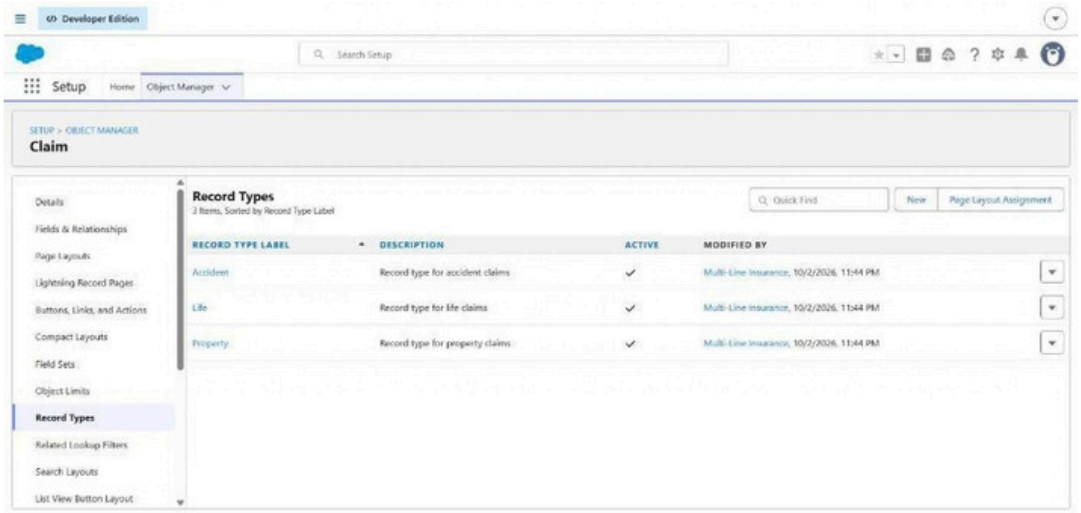
Policy – Fields & Relationships (Policy State picklist)

Create Claim Object and Record Types

- Object Manager →Create → Custom Object. Label: Claim; Plural:Claim; Record Name: Claim Number; Auto Number; Format: C-{0000}; Starting Number 1.
- Allow Reports, Activities, Track Field History; Deployed; Allow Search → Save.
- Claim__c → Record Types → New: Accident, then Property and Life (same steps as the Policy record types).



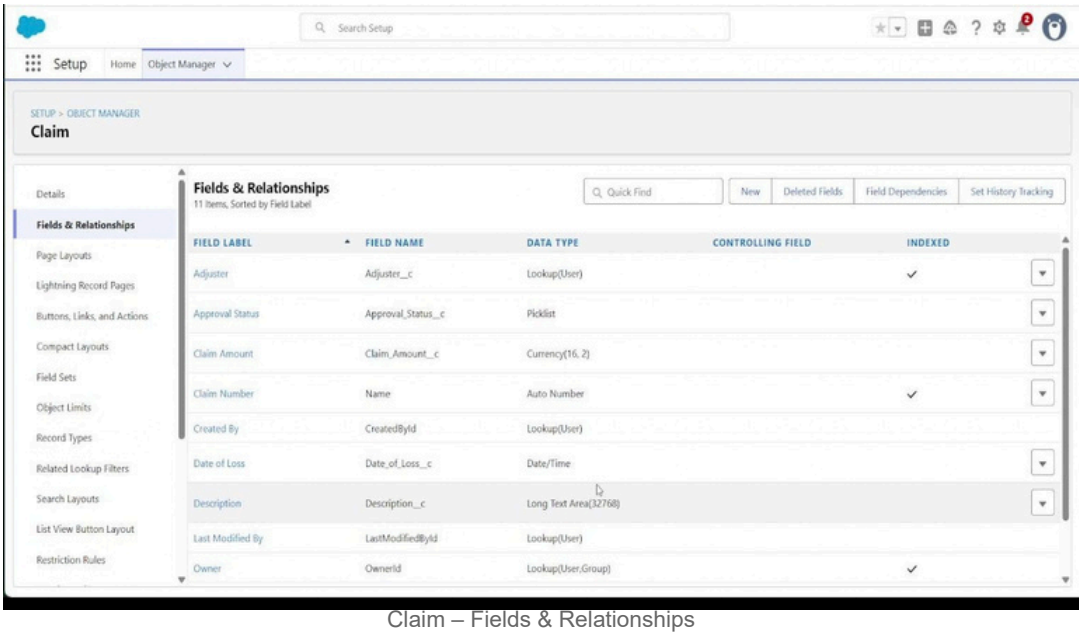
New Custom Object – Claim



Claim record types: Accident, Life, Property

Fields in the Claim Object

S.No	Field Name	DataType
1	Adjuster	Lookup(User)
2	Approval Status	Picklist(New, Submitted for Approval, Approved, Rejected)
3	Claim Amount	Currency(16,2)
4	Date of Loss	Date/Time
5	Description	Long Text Area
6	Policy	Lookup(Policy)
7	Policy Account Holder State (used for state sharing)	Text(10)

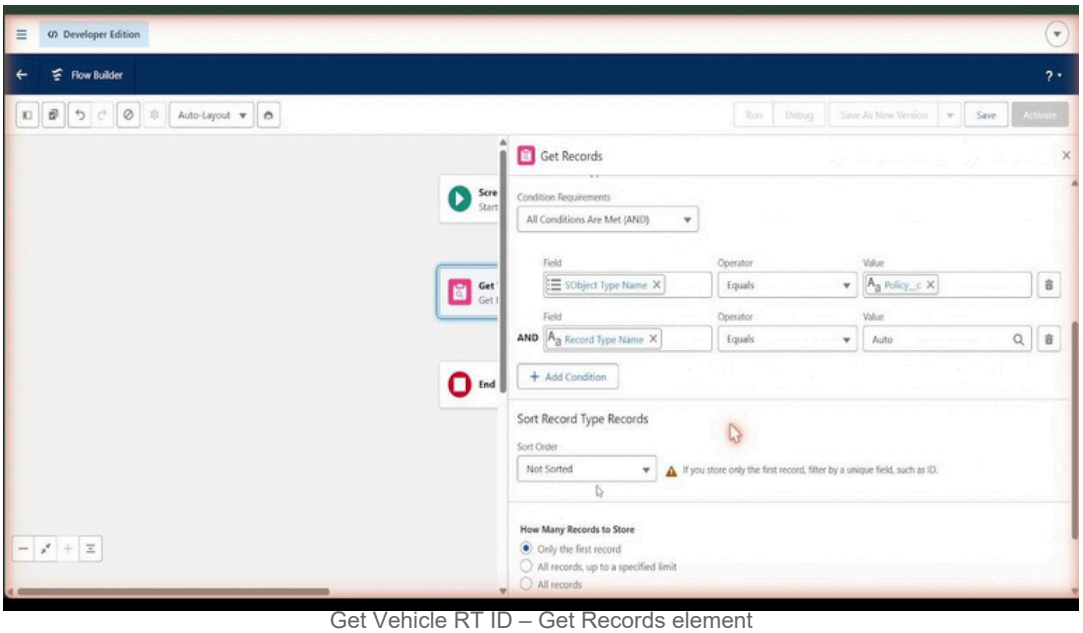


Claim – Fields & Relationships

AutoQuoting Screen Flow

Step 1 – Create the flow and get the record type

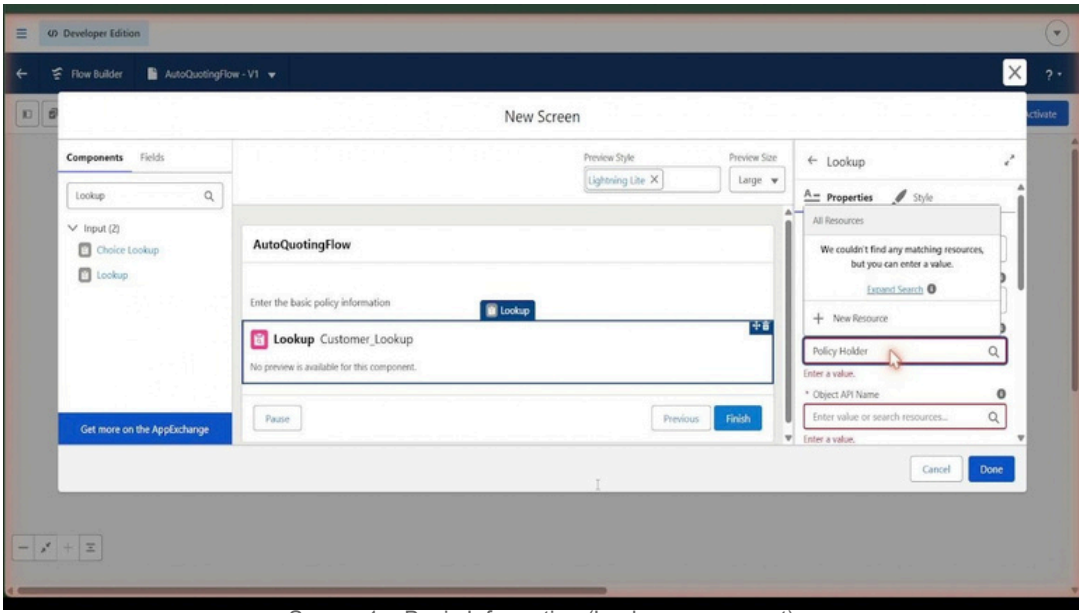
- Setup → Flows→ New Flow →Screen Flow→Create.
- Add Get Records: label Get Vehicle RT ID; object Record Type; conditions SobjectType = Policy__c AND Name = Auto.



Get Vehicle RT ID – Get Records element

Step 2 – Screen 1: Policy basics

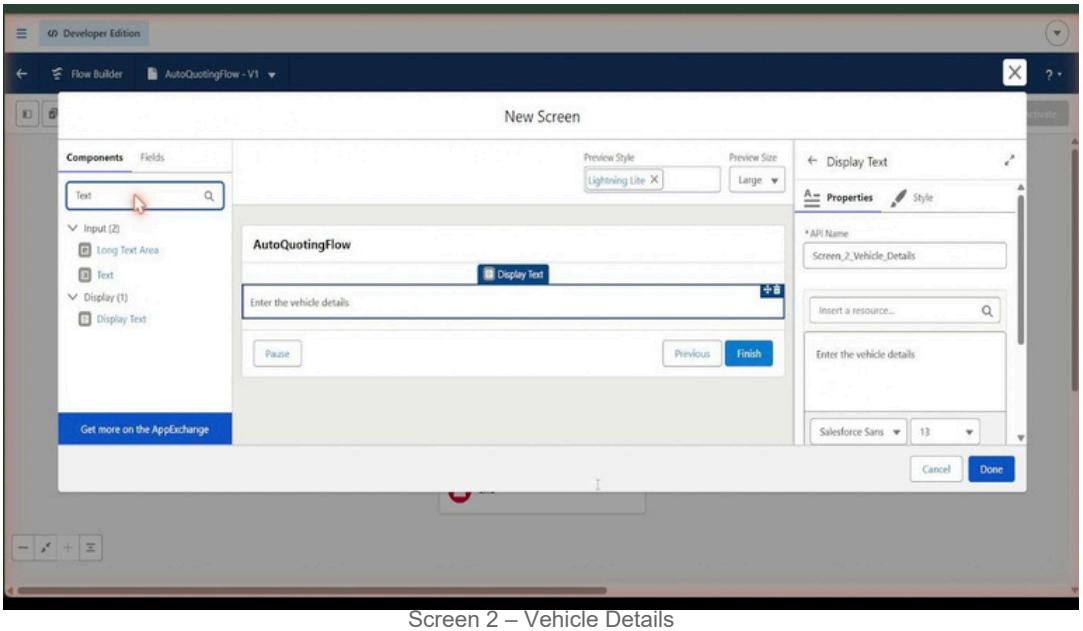
- Screenlabel: NewAutoPolicyQuote– Basic Information (Screen_1_Policy_Basics).
- Components: Lookup (Customer__c,label Policy Holder), Date (Policy Start Date), Picklist (Policy State using a picklist choice set).



Screen 1 – Basic Information (Lookup component)

Step 3 – Screen 2: Vehicle details

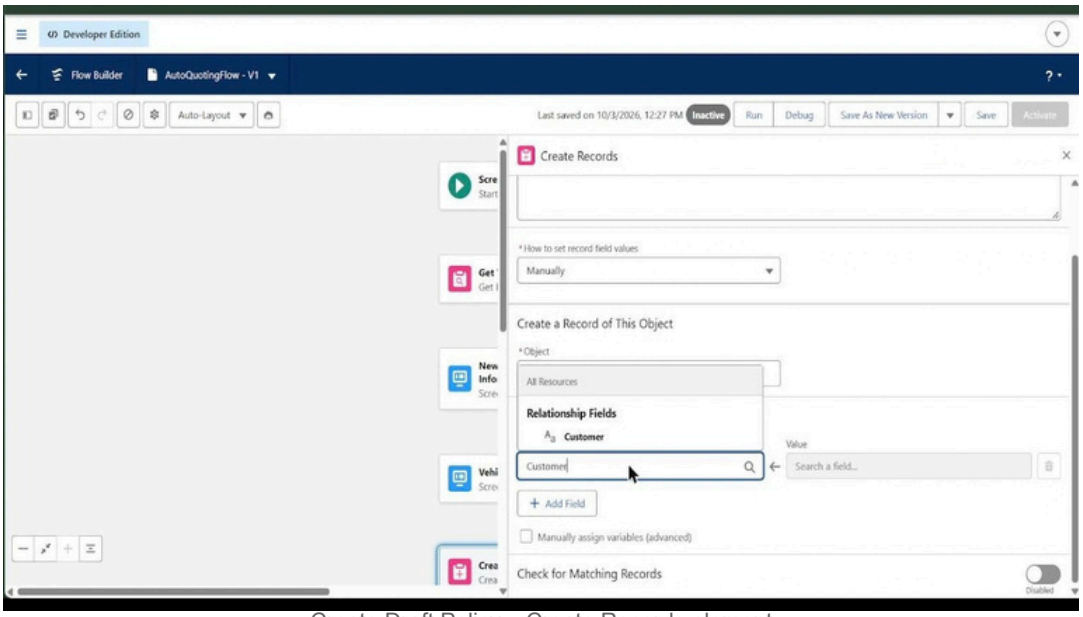
- Screenlabel:Vehicle-SpecificDetails(Screen_2_Vehicle_Details) with text inputs VIN and Model Year.



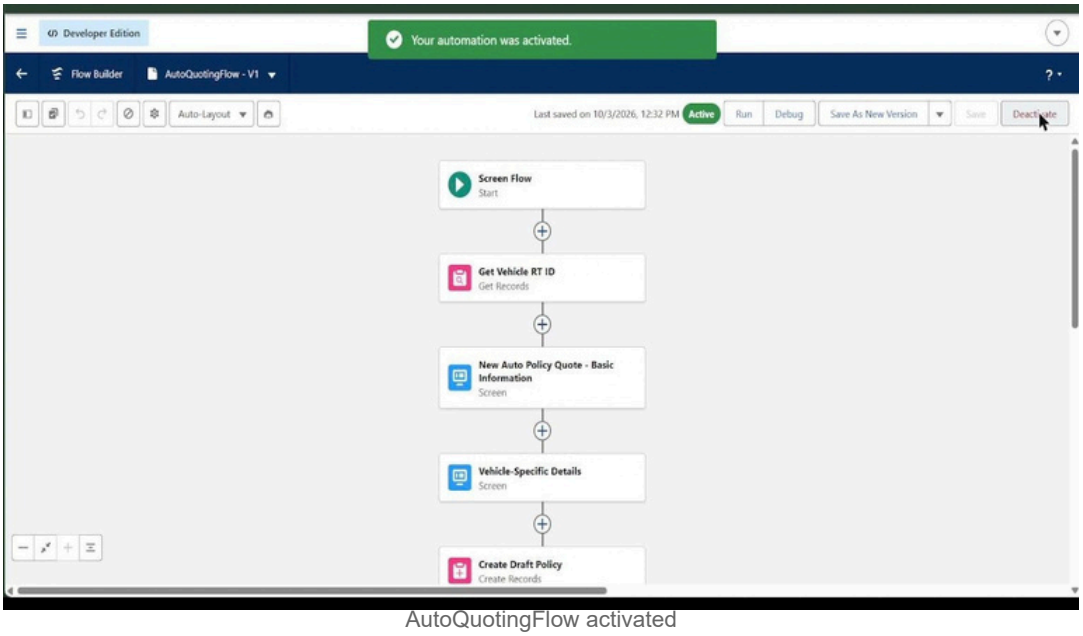
Screen 2 – Vehicle Details

AutoQuoting Flow – Create Draft Policy

- Add Create Records: label Create Draft Policy; Policy object; store the new Idinvariable varPolicyId.
- Map Customer, Model Year, Policy Start Date, Policy State, VIN and RecordTypeId from the screens / Get Vehicle RT ID.
- Add the Calculate Premium Apex action and an Update Records element to store the premium (Milestone 2).
- Save as AutoQuotingFlow and Activate.



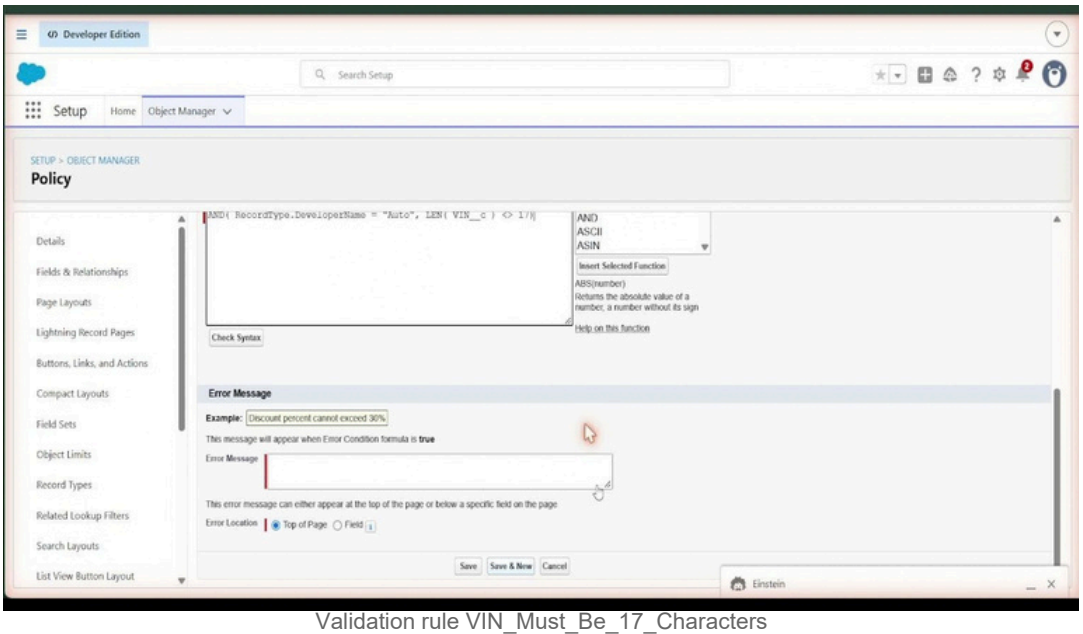
Create Draft Policy – Create Records element



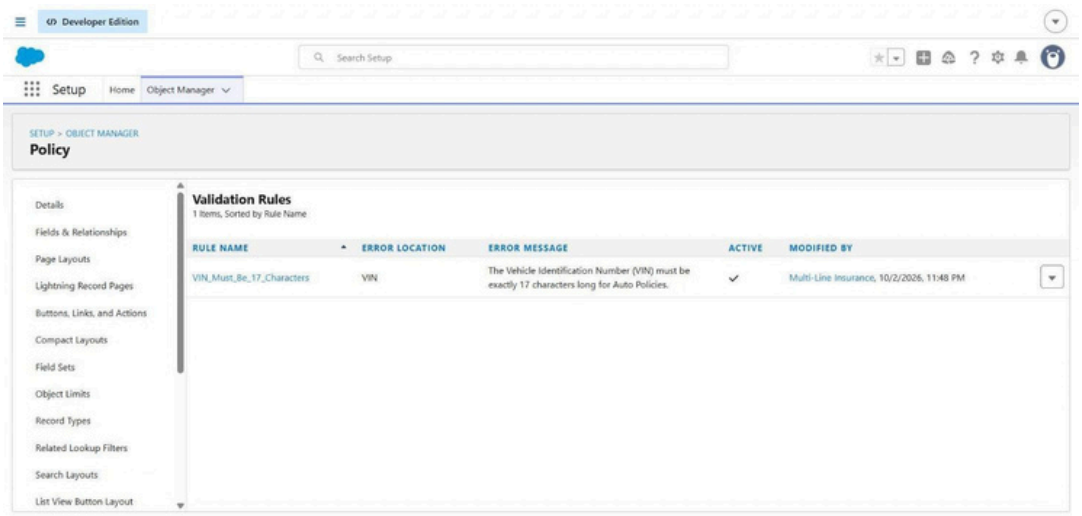
AutoQuotingFlow activated

Validation Rule on Policy Object

- Object Manager → Policy→ValidationRules → New.
- Rule Name: VIN_Must_Be_17_Characters; Active.
- Formula: AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)
- Error message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies. Location: Field → VIN.



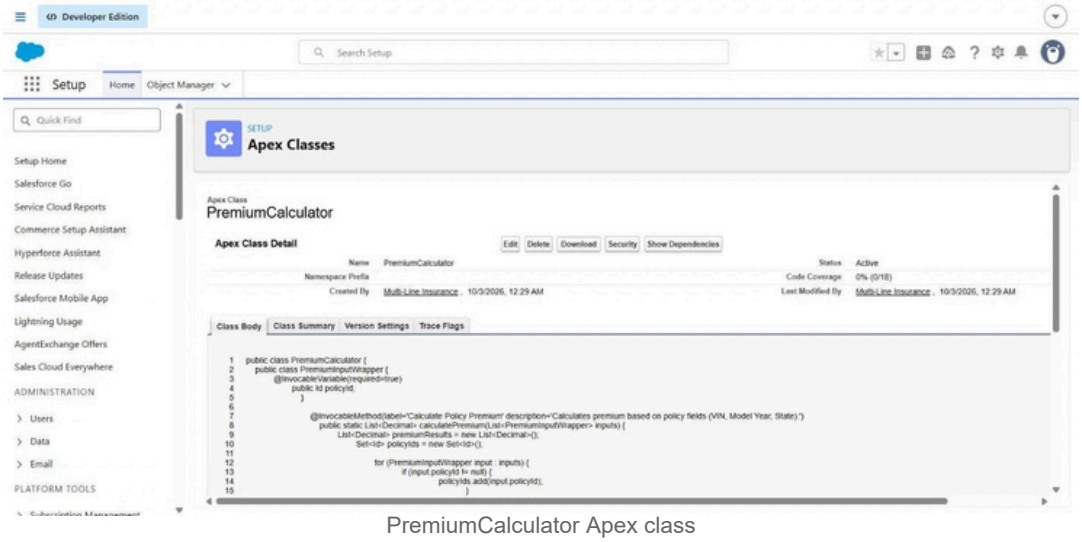
Validation rule VIN_Must_Be_17_Characters



Validation rule is active on the Policy object

Milestone 2 – Apex Class PremiumCalculator

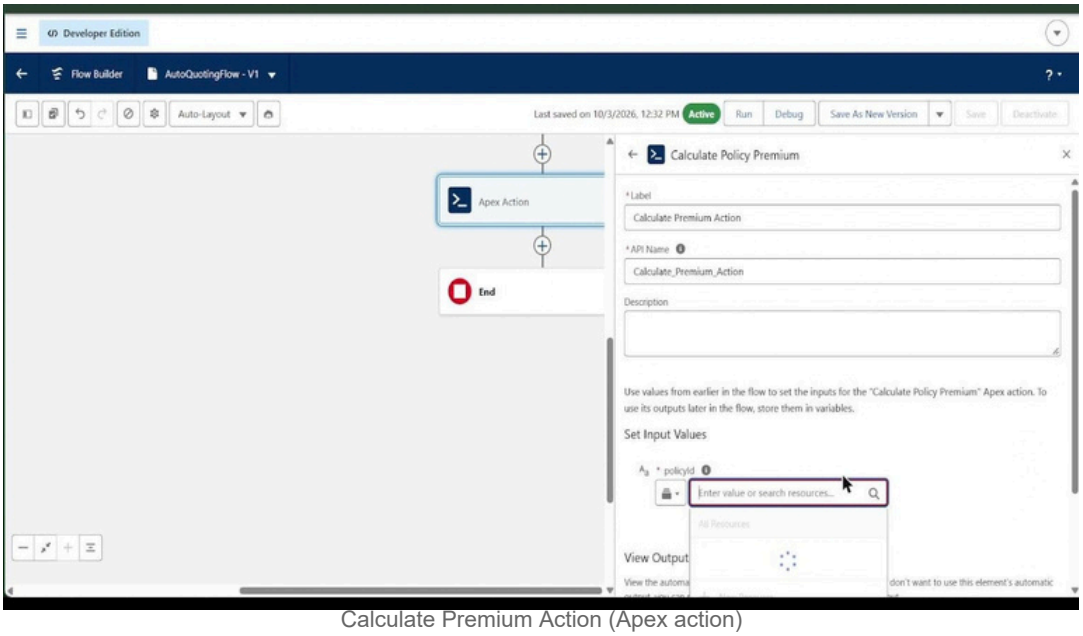
- Setup (gear)→DeveloperConsole → File → New→ Apex Class→ name PremiumCalculator.
- The class exposes an @InvocableMethod 'Calculate Policy Premium' that accepts a list of PremiumInputWrapper (policyId) and returns the calculated premium so flows can call it.
- Save the class (Ctrl+S).



PremiumCalculator Apex class

Call it from the AutoQuotingFlow

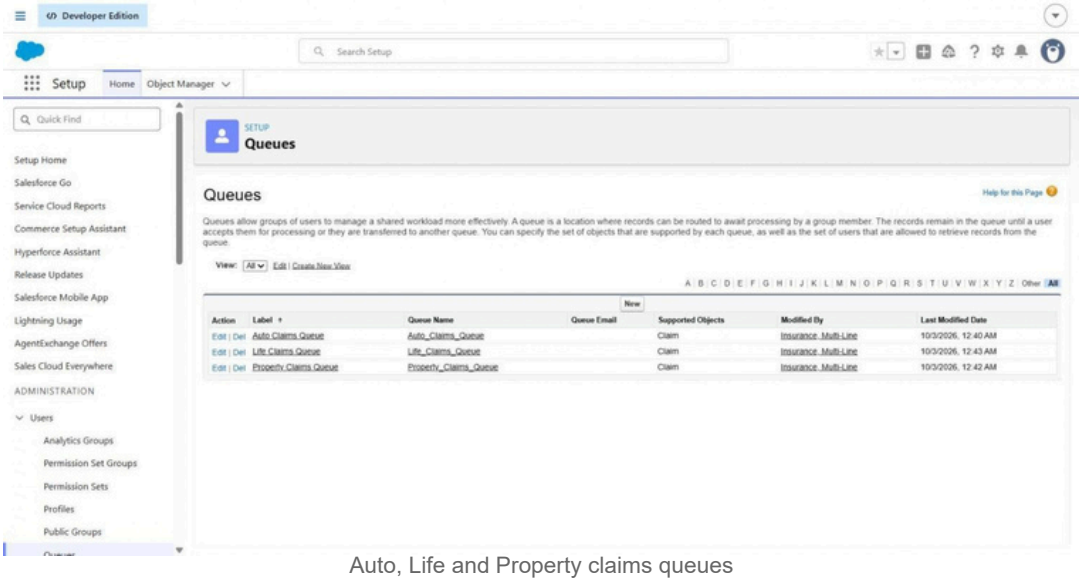
- Add an Action element 'Calculate Premium Action' (Apex action) after Create Draft Policy, passing varPolicyId; store the result in varCalculatedPremium.
- Add 'Update Policy With Premium' to write the value to Premium__c.



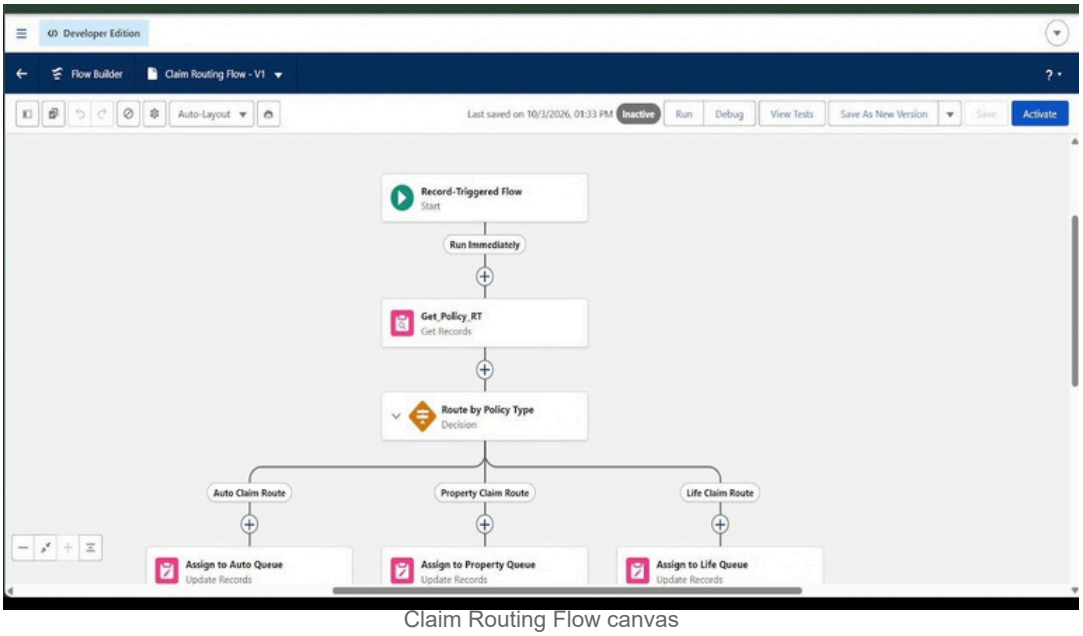
Calculate Premium Action (Apex action)

Claim Routing Flow and Queues

- Setup → Queues →New: Auto Claims Queue, Property Claims Queue,Life Claims Queue (supported object: Claim).
- Create a Record-Triggered Flow named Claim Routing Flow on Claim (after save, record created).
- Get Records Get_Policy_RT to find the policy's record type; Decision 'Route by Policy Type'.
- Update Records to set OwnerId to the Auto / Property / Life queue. Save and Activate.



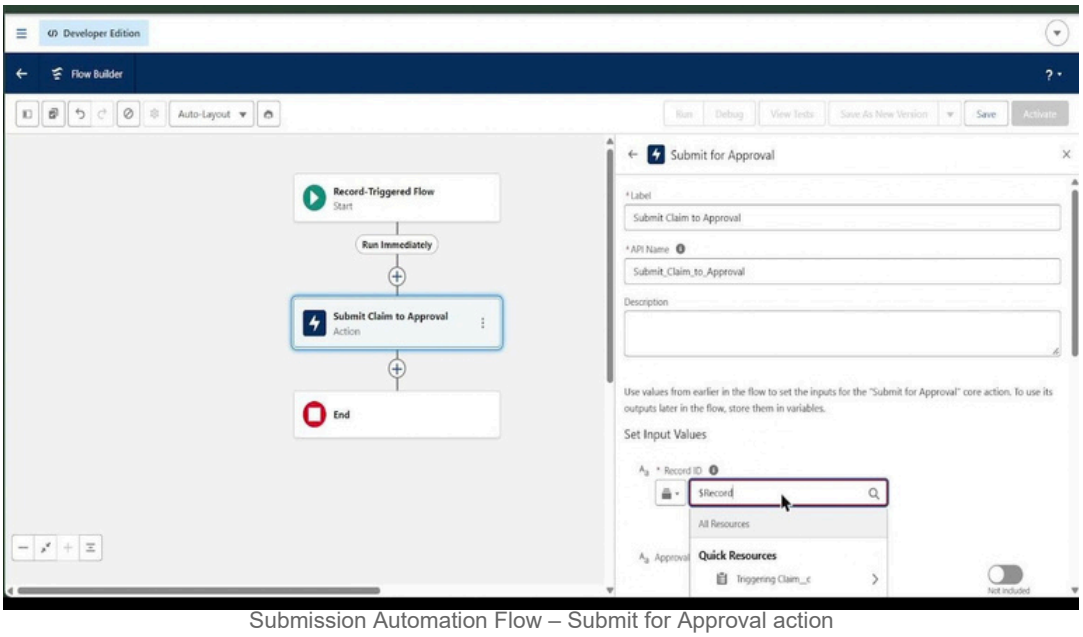
Auto, Life and Property claims queues



Claim Routing Flow canvas

Submission Automation Flow

- Record-Triggered Flow on Claim (createdorupdated)withentrycondition Claim Amount > 50000 AND Approval Status = New.
- Action: Submit for Approval using the High_Value_Claim_Approval process.
- Save as Submission Automation Flow and Activate.

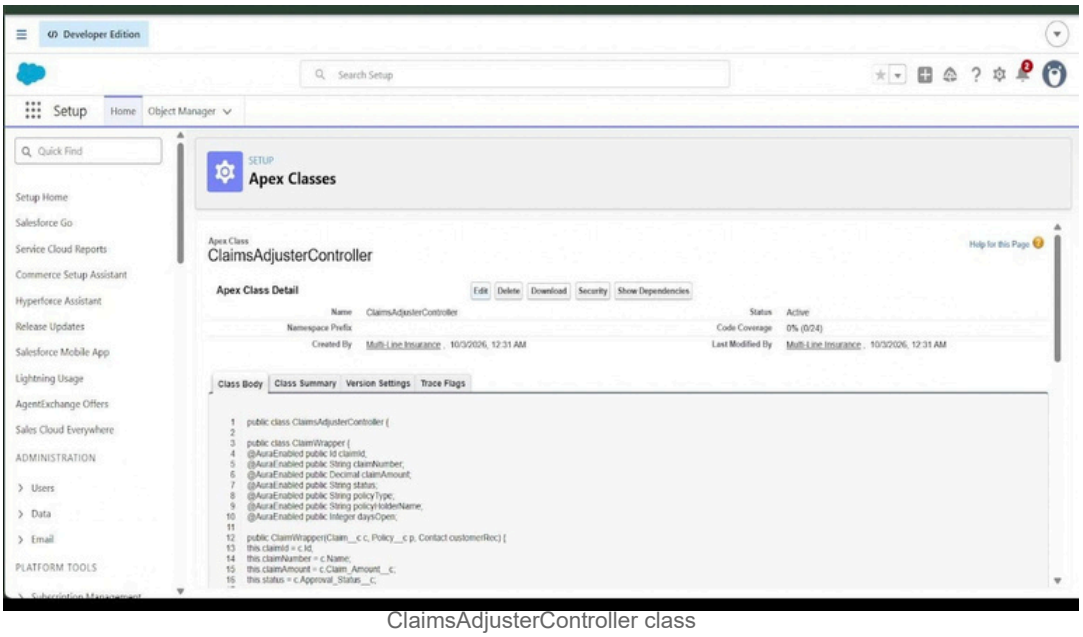


Submission Automation Flow – Submit for Approval action

Milestone 3 – ClaimsAdjusterController (Apex)

- DeveloperConsole → NewApexClass→ ClaimsAdjusterController.
- getAssignedClaims() is @AuraEnabled(cacheable=true); it queries Claim__c records owned by the running user (UserInfo.getUserId()) along with Policy record type and policy holder name, and

returns a list of ClaimWrapper objects (claimId, claimNumber, claimAmount, status, policyType, policyHolderName, daysOpen).



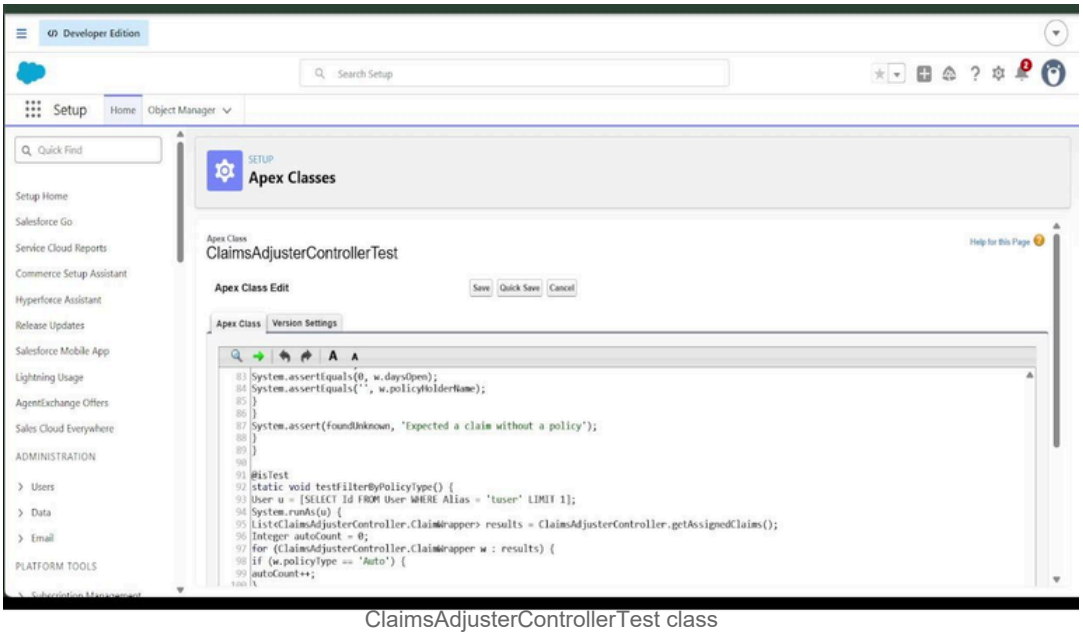
ClaimsAdjusterController class

Claims Dashboard LWC

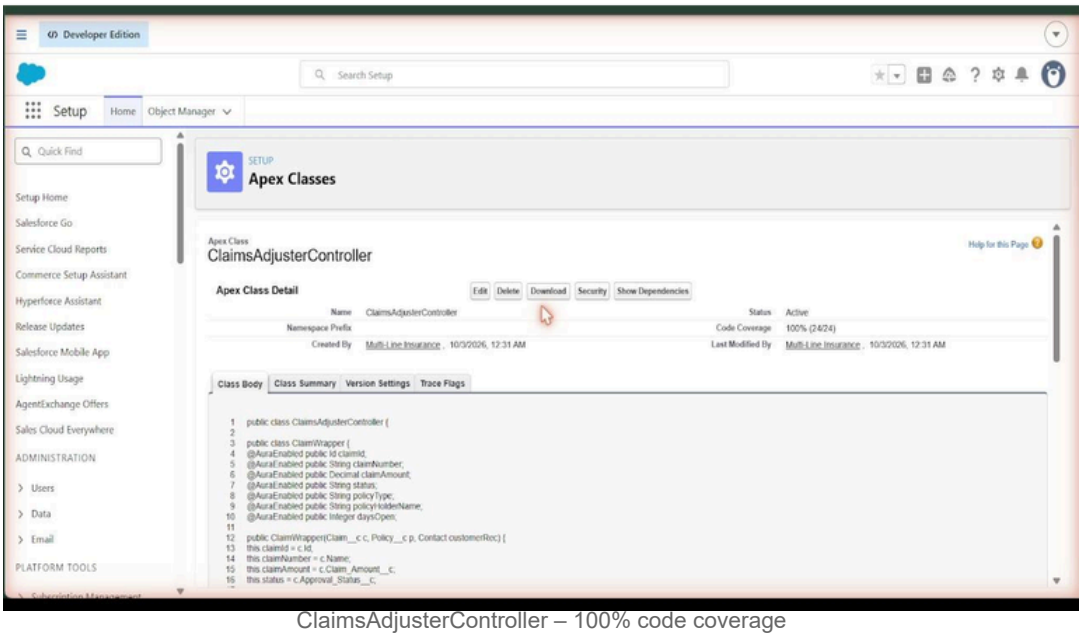
- Create the Lightning Web Components claimsDashboardLwc (parent) and claimTileLwc (child).
- claimsDashboardLwc wires getAssignedClaims, shows a combobox filter (All / Auto / Property / Life) and renders one <c-claim-tile-lwc> per visible claim.
- claimTileLwc receives claimData via @api and shows an icon per policy type (Auto → truck, Property → home, Life → connected_apps), claim number, policy holder, amount, status and days open.
- Deploy both components to the org (Lightning Studio / VS Code).

Apex Test Class – ClaimsAdjusterControllerTest

- Developer Console → New Apex Class → ClaimsAdjusterControllerTest.
- @testSetup creates a Standard User, assigns the Claims_Adjuster_Access permission set, and creates a Contact, an Auto Policy and Claims owned by that user.
- Test methods cover data retrieval, a claim without a policy, policy-type filtering, and a user with no claims.
- Run the test class: ClaimsAdjusterController reaches 100% code coverage (requirement: 95%+).



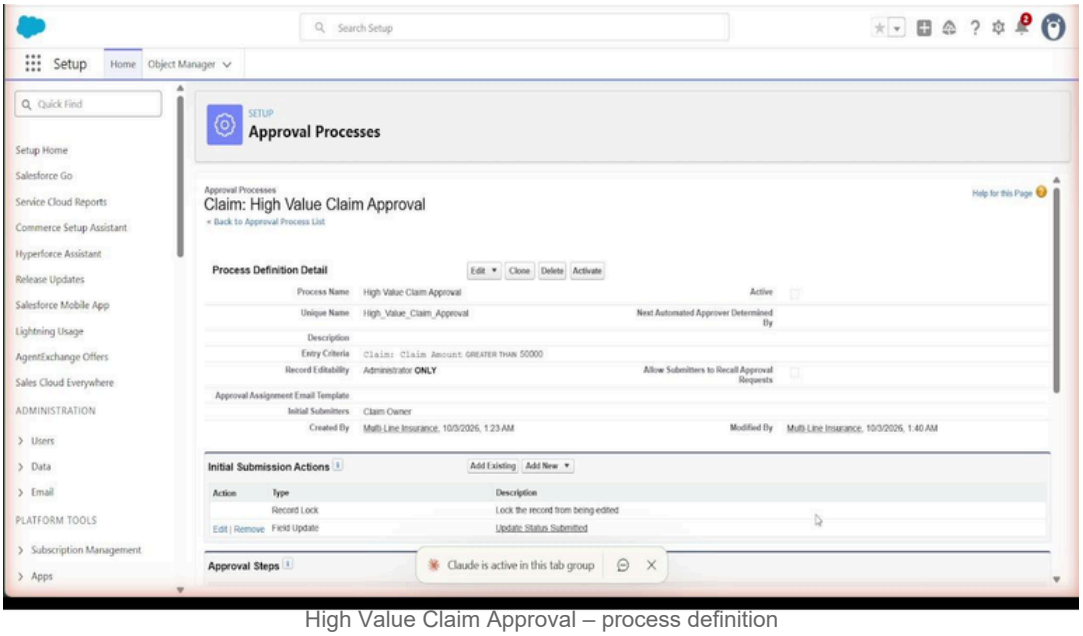
ClaimsAdjusterControllerTest class



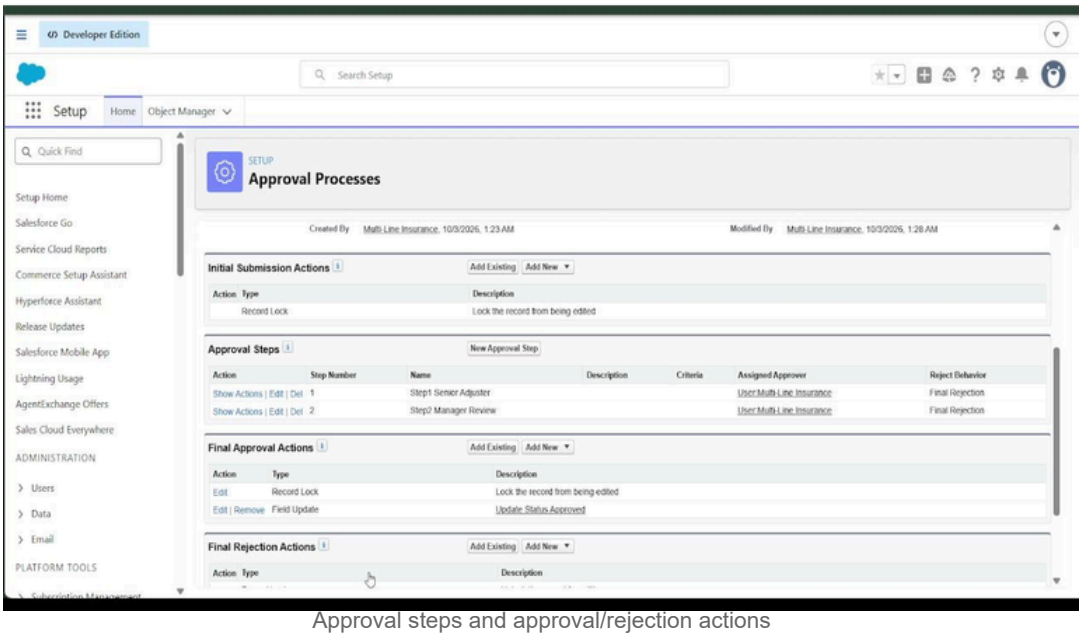
ClaimsAdjusterController – 100% code coverage

Milestone 4 – Approval Process for High-Value Claims

- Setup → Approval Processes → Claim→ StandardSetupWizard / Jump StartWizard.
- Name: High Value Claim Approval; entry criteria Claim Amount > 50000.
- Step 1: Senior Adjuster; Step 2: Manager Review (sequential).
- Field updates: Update Status Submitted (initial submission), Approved (final approval), Rejected (final rejection).
- Activate the process.



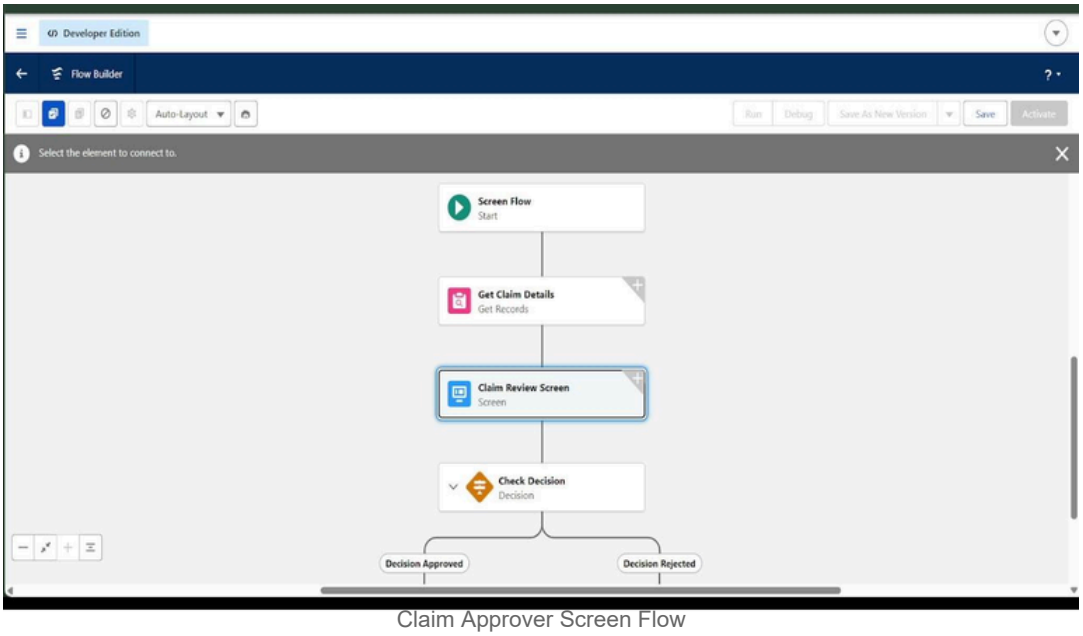
High Value Claim Approval – process definition



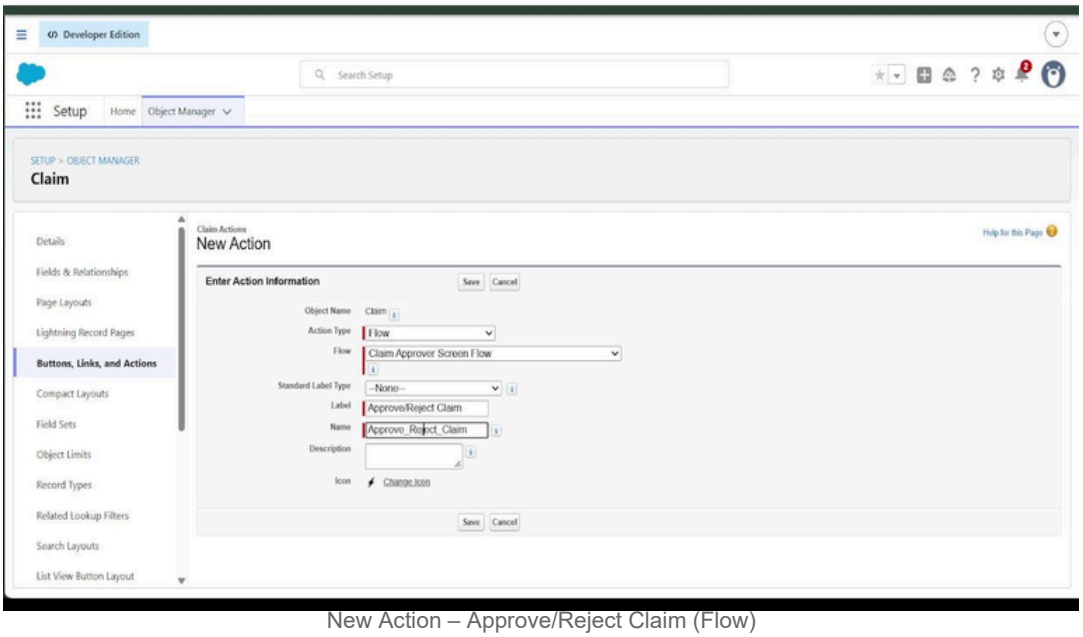
Approval steps and approval/rejection actions

Claim Approver Screen Flow and Quick Action

- Create Screen Flow'Claim Approver Screen Flow'withinput variable recordId; GetClaim Details; screen with radio buttons Approve / Reject and Approver Comments.
- Decision 'Check Decision' → Approve Claim / Reject Claim (Submit for Approval actions).
- Setup → Object Manager → Claim → Buttons, Links and Actions → New Action: type Flow, flow Claim Approver Screen Flow, label Approve/Reject Claim.
- Page Layouts → Claim Layout → add the action to Salesforce Mobile and Lightning Experience Actions → Save.



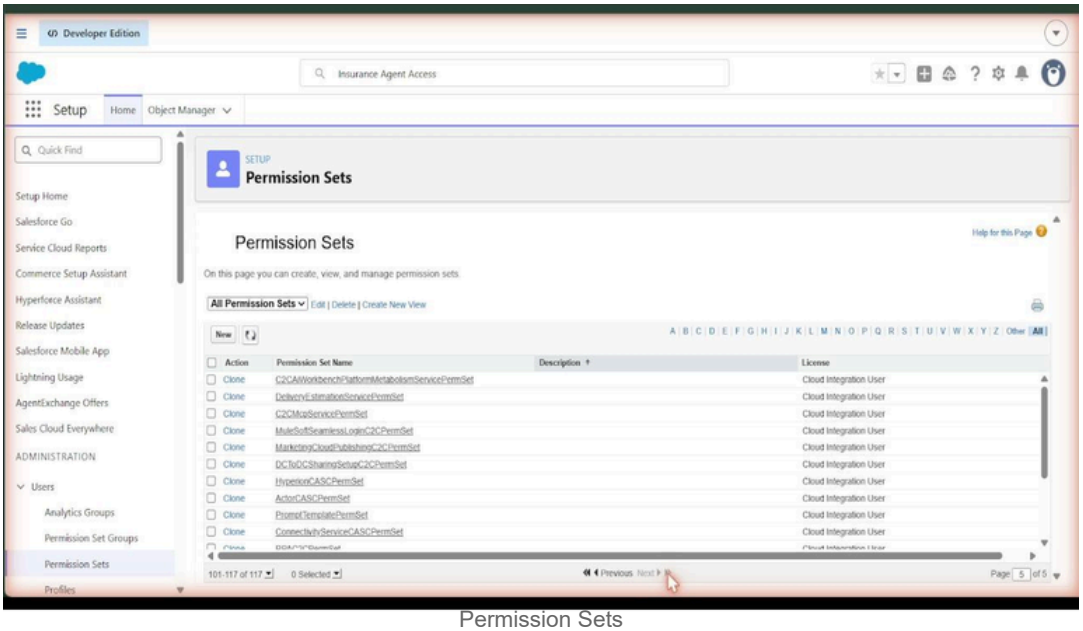
Claim Approver Screen Flow



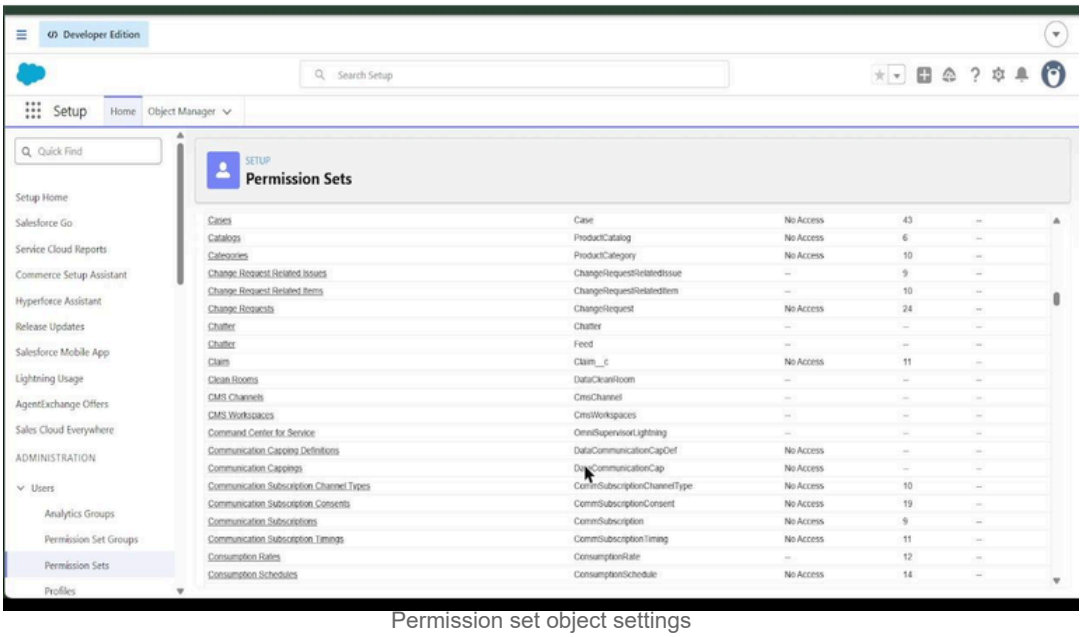
New Action – Approve/Reject Claim (Flow)

Permission Sets

Permission Set	Access
InsuranceAgent Access	Policy – Create, Read; Contact – Read, Edit; Claim – Read; PremiumCalculator Apex class enabled
Claims Manager Access	Policy – Read; Contact – Read, Edit; Claim – Read, Edit, Delete; Run Reports
Claims Adjuster Access	Policy – Read; Claim – Read, Edit; ClaimsAdjusterController Apex class enabled



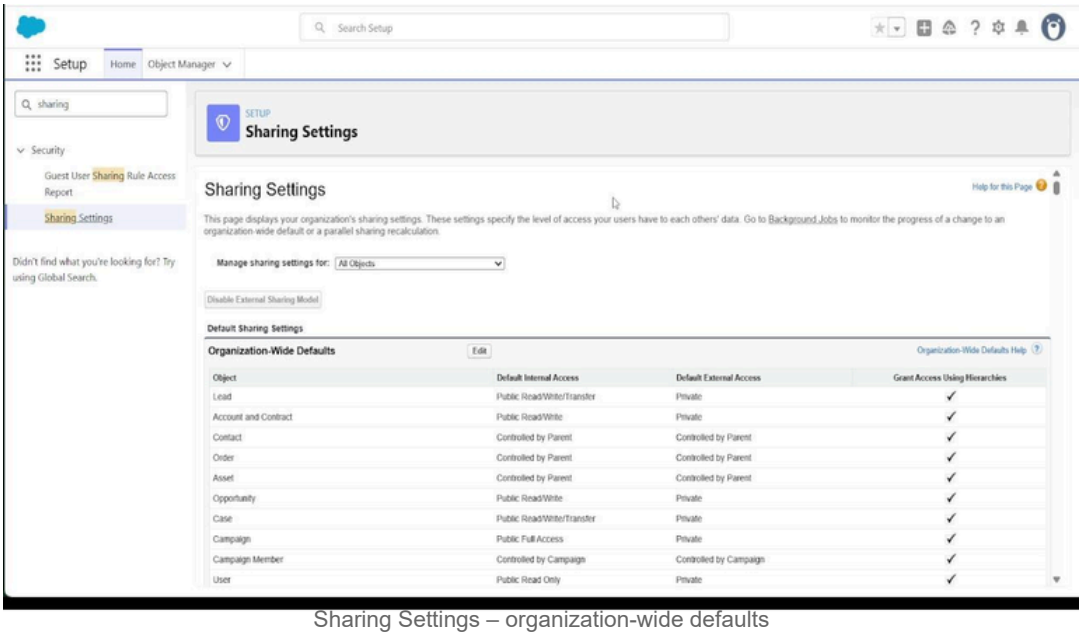
Permission Sets



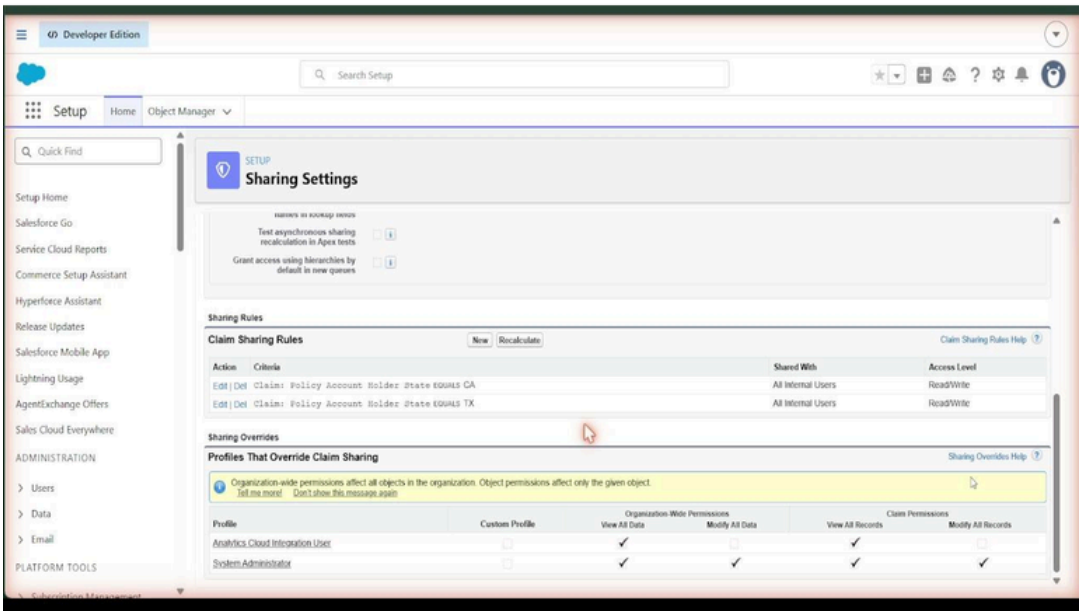
Permission set object settings

Sharing Rules – Adjusters See Claims by State

- Setup → Sharing Settings → set the Claim default internal access to Private.
- Create Record-Triggered Flow 'Claim Policy Holder State Update' (before save): get the Policy of the claim and set Policy Account Holder State on the Claim from the Policy State.
- Sharing Settings → Claim Sharing Rules → New.
- Rule 1: 'Share Records based on Policy State CA' – based on criteria: Policy Account Holder State equals CA; share with All Internal Users; Read/Write.
- Rule 2: 'Share Records based on Policy State' – criteria: Policy Account Holder State equals TX; All Internal Users; Read/Write.



Sharing Settings – organization-wide defaults



Claim sharing rules for CA and TX

Testing the Solution

- Run AutoQuotingFlow and createan Auto policy quote;verifythepremium is filled in.
- Enter a VIN that is not 17 characters and confirm the validationmessage.
- Create a claim and confirm it is routed to the correct queue.
- Create a claim above 50,000 and confirm it is submitted for approval and the status is updated after Step 1 and Step 2.
- Open the Claims Dashboard and verify claim tiles and the policy-type filter.